LAPORAN
WORKSHOP SISTEM INFORMASI
WEB SERVER
(TIF330805)
SEMESTER III



Acara 17

Nama:

Nila Agustin Prima Aurelia

NIM:

E41250666

Golongan:

A

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)

TEKNIK INFORMATIKA KAB. NGANJUK

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI JEMBER

TAHUN 2026

Bab 1. Pendahuluan

1.1 Latar Belakang

Aplikasi web modern perlu bertukar data dengan aplikasi lain, misalnya aplikasi mobile atau layanan pihak ketiga. Application Programming Interface (API) menjadi jembatan komunikasi tersebut, dan JSON menjadi format pertukaran data yang paling umum karena ringan serta mudah diproses oleh berbagai bahasa pemrograman.

Pada Acara 16, API CRUD data mahasiswa dibangun dalam satu berkas (api/mahasiswa.php) yang memuat koneksi database, validasi, query, dan pembentukan response sekaligus. Cara ini sederhana, tetapi sulit dirawat ketika endpoint bertambah. Acara 17 melanjutkannya dengan memisahkan tanggung jawab kode ke dalam pola Model-View-Controller (MVC), dilengkapi Router, autoloader, dan Front Controller sebagai satu-satunya titik masuk aplikasi. Struktur ini menjadi fondasi bagi acara-acara berikutnya.

Praktikum ini memakai database si_akademik yang sudah dibuat pada acara sebelumnya (tabel mahasiswa, prodi, matakuliah) sehingga data dan relasinya tetap konsisten. Tabel buku ditambahkan pada database yang sama untuk keperluan tugas mandiri.

1.2 Tujuan Praktikum

1. Menyusun struktur folder proyek PHP Native berpola MVC.
2. Membangun koneksi database PDO dalam bentuk class (Singleton sederhana).
3. Membuat Model yang memuat seluruh query dengan prepared statement.
4. Membuat Controller yang menerima request dan mengembalikan response JSON.
5. Menerapkan autoloader (spl_autoload_register), Router, dan Front Controller (.htaccess).
6. Menguji endpoint CRUD menggunakan Postman.
7. Mengembangkan resource baru (Buku), validasi NIM duplikat, dan dokumentasi endpoint.

Bab 2. Dasar Teori

2.1 API dan PHP Native

API adalah kumpulan aturan dan mekanisme yang memungkinkan dua aplikasi bertukar data tanpa mengetahui implementasi internal masing-masing. Pada web, API umumnya memakai protokol HTTP dengan format data JSON. PHP Native berarti kode PHP murni tanpa framework, sehingga alur kerja API (menerima request, memproses logika, mengakses database, mengirim response) dapat dipahami secara langsung.

2.2 CRUD dan HTTP Method

CRUD adalah empat operasi dasar pengelolaan data. Setiap operasi dipetakan ke HTTP method tertentu.

2.3 Arsitektur MVC, Autoloader, dan Front Controller

MVC memisahkan kode menjadi tiga lapisan. Model mengelola data dan berinteraksi langsung dengan database. View menyajikan tampilan kepada pengguna. Controller menerima request, memanggil Model, lalu menentukan response yang dikembalikan. Pemisahan ini umumnya diterapkan dengan pemrograman berorientasi objek (OOP).

Fungsi bawaan spl_autoload_register() mendaftarkan autoloader yang dipanggil otomatis saat sebuah class dipakai tetapi belum dimuat, sehingga tidak perlu menulis require satu per satu. Pola Front Controller menjadikan satu berkas (public/index.php) sebagai titik masuk tunggal; seluruh request diteruskan ke Controller yang sesuai melalui proses routing. Penempatan folder app/ di luar public/ juga meningkatkan keamanan, karena kode sumber dan kredensial database tidak dapat diakses langsung dari browser.

2.4 Format JSON

JSON (JavaScript Object Notation) adalah format pertukaran data yang ringan dan mudah dibaca manusia maupun mesin. Di PHP, data diubah menjadi JSON dengan json_encode(), sedangkan JSON dari client dibaca dengan file_get_contents('php://input') yang kemudian didekode dengan json_decode().

2.5 PDO dan Prepared Statement

PDO (PHP Data Objects) menyediakan antarmuka seragam untuk mengakses berbagai basis data. Prepared statement memisahkan struktur perintah SQL dari data yang dimasukkan pengguna, sehingga nilai diikat melalui placeholder (? atau :nama) dan tidak disisipkan langsung ke teks SQL. Teknik ini mencegah celah keamanan SQL Injection.

2.6 Alur Kerja API CRUD

Client -> HTTP Request -> PHP Native API -> MySQL Database

Client <- JSON Response <- PHP Native API <- MySQL Database

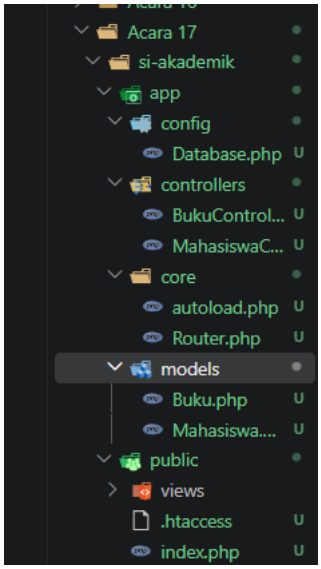
Bab 3. Hasil Praktikum

3.1 Prosedur Kerja

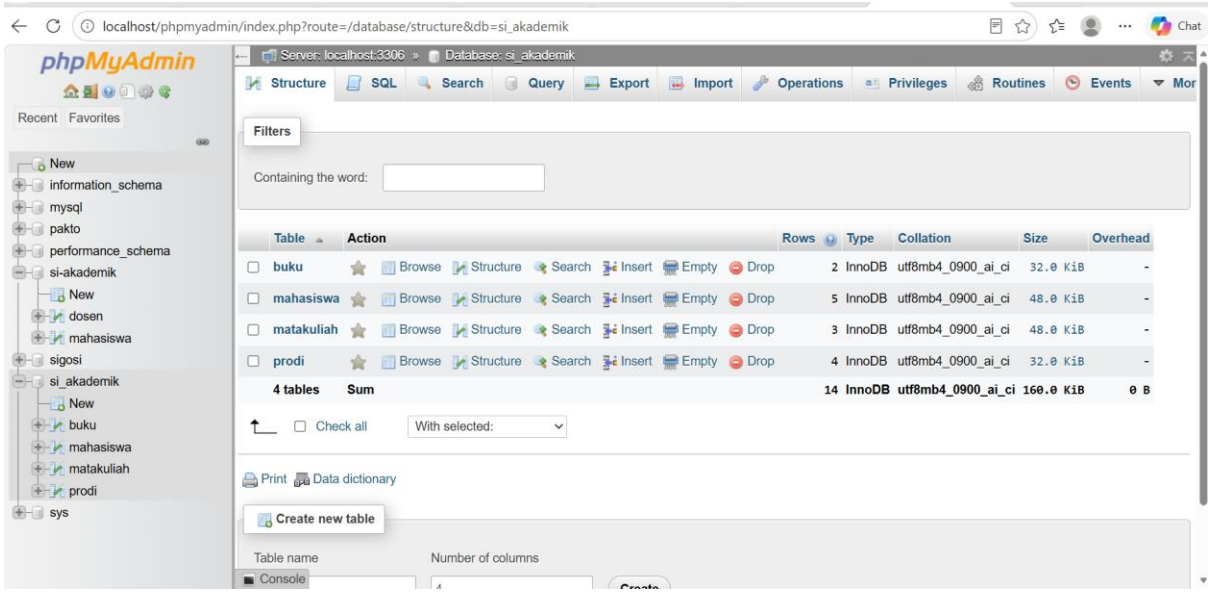
a. Persiapan Lingkungan Kerja

b. Struktur Folder Proyek Berpola MVC

Folder proyek dibuat dengan struktur berikut. Folder public/ adalah satu-satunya folder yang diakses browser, sedangkan app/ berada di luar public/.



c. Database



d. Koneksi Database (app/config/Database.php)

Koneksi dibangun sebagai class sehingga satu koneksi dipakai berulang oleh semua Model (Singleton sederhana). Mode error diatur ke exception dan fetch mode ke array asosiatif.

e. Model Mahasiswa (app/models/Mahasiswa.php)

Seluruh perintah SQL berada di lapisan ini dan memakai prepared statement. Method nimExists() dipakai untuk validasi NIM duplikat (Tugas Mandiri 2), dan prodiExists() memastikan prodi yang dipilih tersedia.

f. Autoloader dan Router

app/core/autoload.php mencari class secara otomatis di folder config, core, models, dan controllers

app/core/Router.php membaca segmen terakhir URL sebagai nama resource, memetakannya ke Controller, lalu memanggil method sesuai HTTP method.

g. Front Controller

public/index.php

Acara 17 > si-akademik > public > index.php

```php
<?php

require_once __DIR__ . '/../app/core/autoload.php';

(new Router())->dispatch();
```

public/.htaccess mengarahkan semua request yang bukan berkas atau folder statis ke index.php:

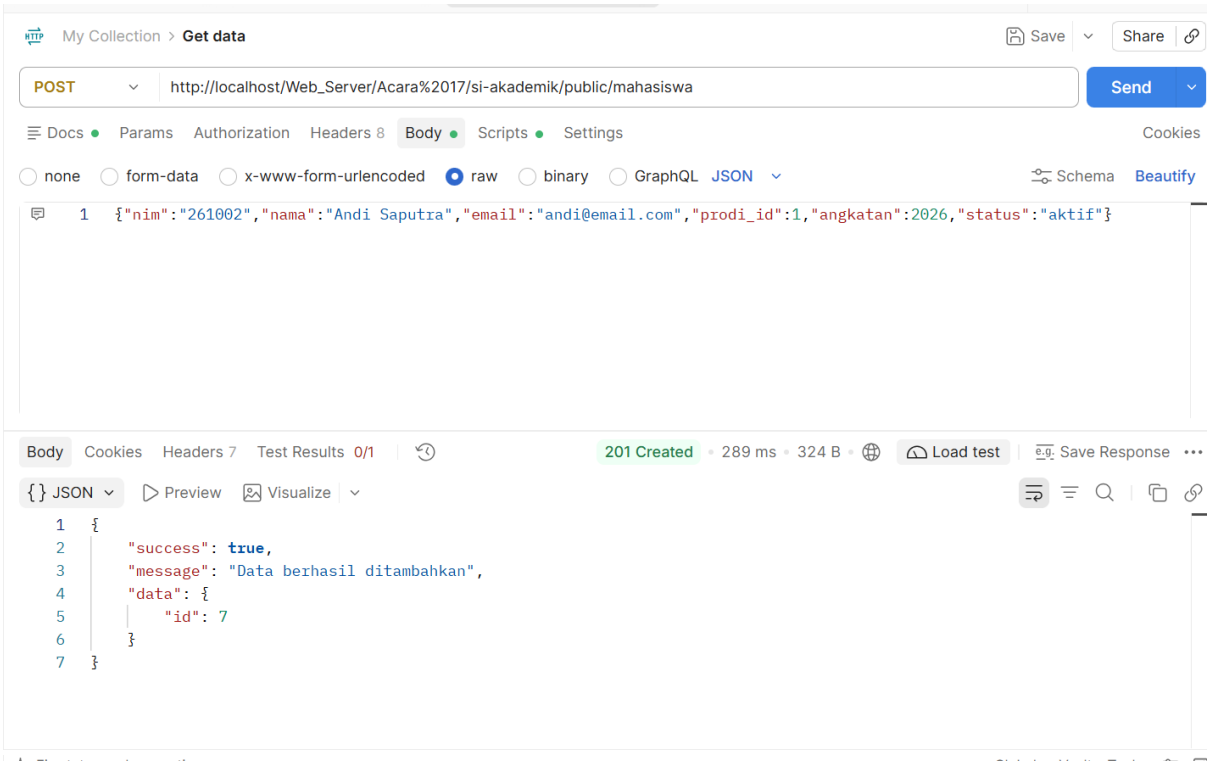
Acara 17 > si-akademik > public > .htaccess

```
RewriteEngine On
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule ^ index.php [QSA,L]
```

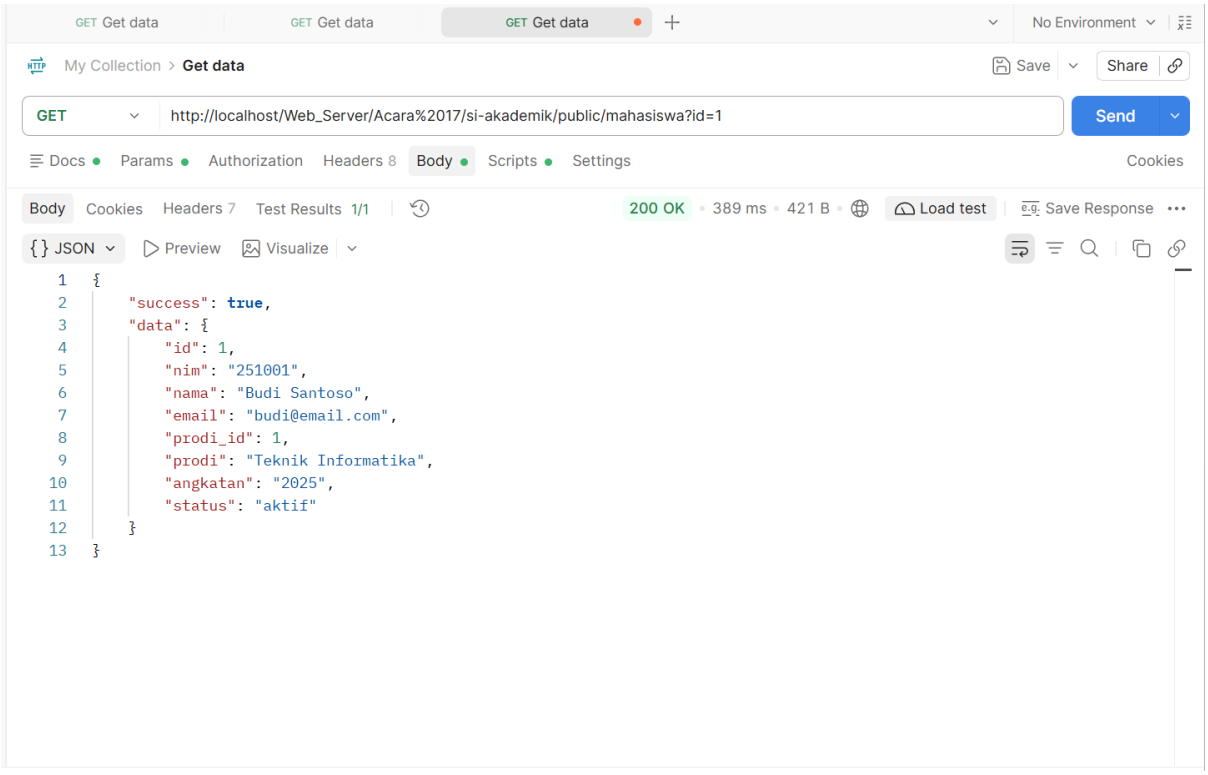
h. Pengujian API dengan Postman

Base URL pengujian: http://localhost/Web_Server/Acara%2017/si-akademik/public. Body dikirim dengan pilihan Body, raw, JSON.

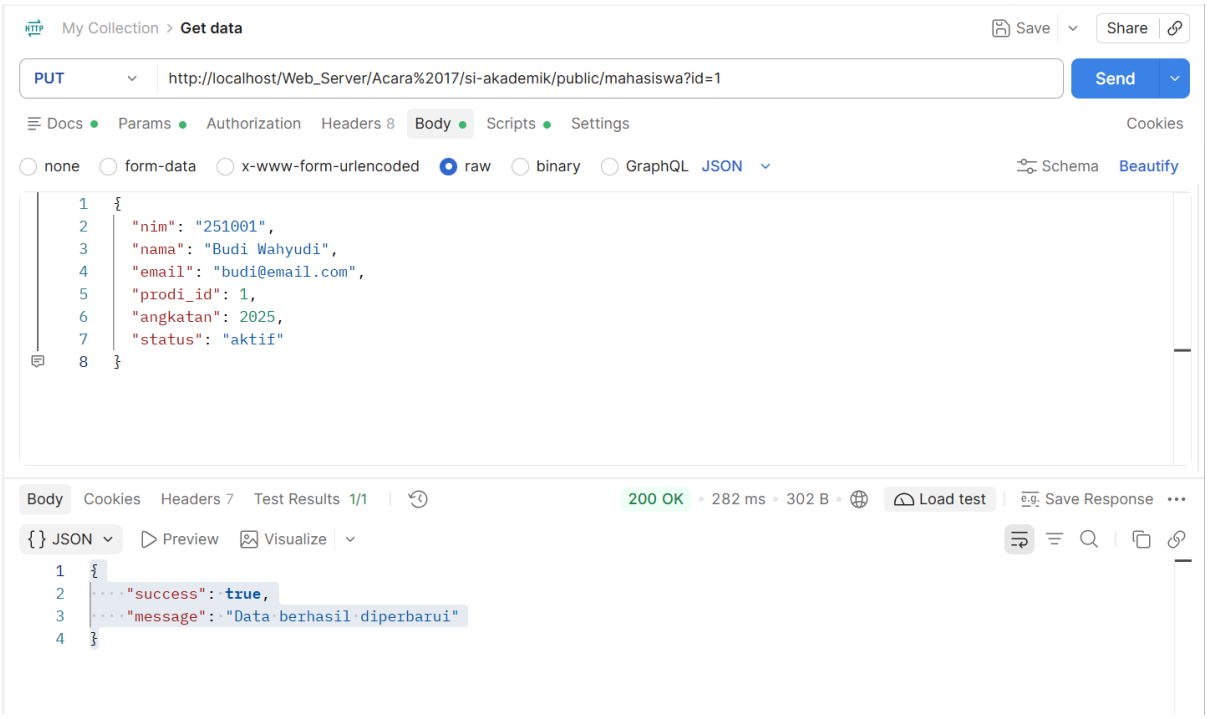
- POST/mahasiswa



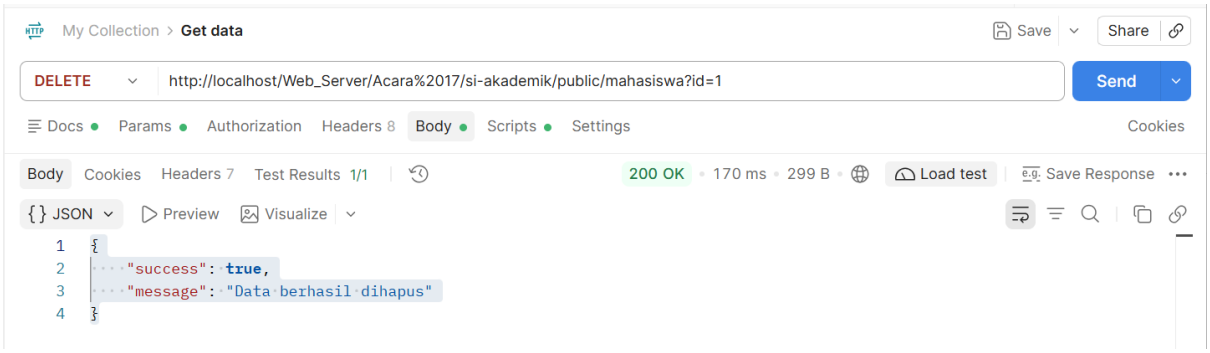
- GET/mahasiswa?id=1



- PUT/mahasiswa?id=1 (mengubah nama)



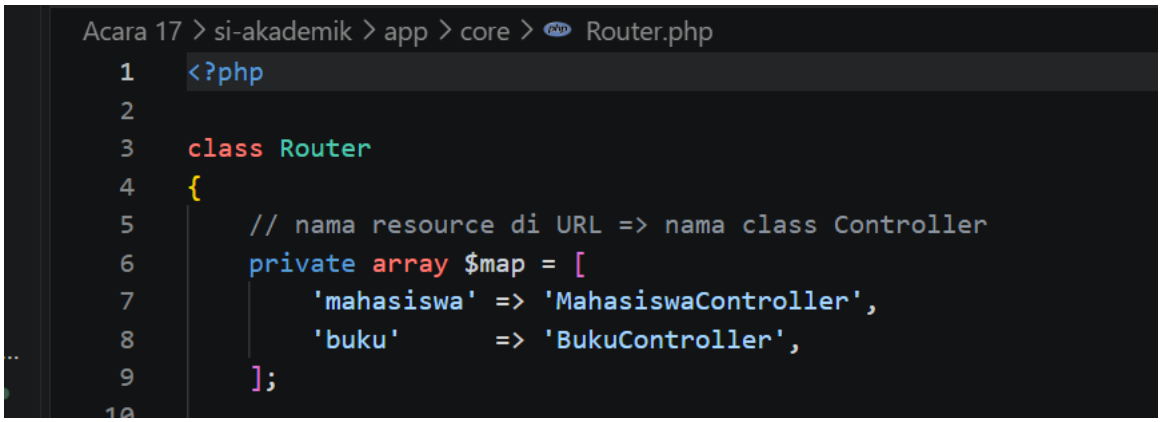
- DELETE/mahasiswa?id=1

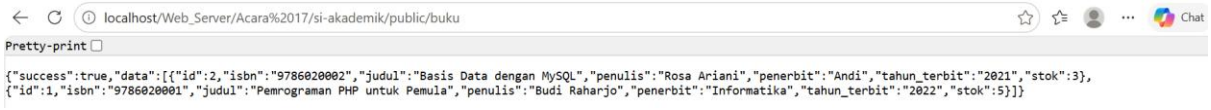


3.2 Tugas Mandiri

a. Model dan Controller Buku

Resource baru dibuat dengan pola MVC yang sama. Karena tabel buku belum ada, tabel dibuat terlebih dahulu pada database si_akademik. Resource buku kemudian didaftarkan pada peta Router (lihat Langkah 7), yaitu entri 'buku' => 'BukuController





b. Validasi NIM tidak boleh duplikat

Validasi diterapkan pada lapisan Model melalui method nimExists(). Method ini mengecek apakah NIM sudah dipakai, dan parameter excludeId membuat NIM milik data yang sedang diubah tidak dihitung sebagai duplikat saat proses update. Controller memanggilnya pada store() dan update() lalu mengembalikan status 409 Conflict. Selain itu, tabel mahasiswa memiliki UNIQUE KEY nim sebagai pengaman kedua di tingkat database.

Pengujian postman:

POST/mahasiswa dengan NIM 251001 (sudah dipakai)

PUT/mahaasiswa?id=2 dengan NIM 251001 (milik mahasiswa lain)

c. Dokumentasi Endpoint dan Diagram Alur

Base URL: http://localhost/Web_Server/Acara%2017/public

Method	Alamat	Request	Response

GET	/mahasiswa		{ "success": **true**, "data": [{ "id": 7, "nim": "261002", "nama": "Andi Saputra", "email": "andi@email.com", "prodi_id": 1, "prodi": "Teknik Informatika", "angkatan": "2026", "status": "aktif" }, { "id": 5, "nim": "261001", "nama": "Ezra Faris", "email": "ezra123@gmail.com", "prodi_id": 1, "prodi": "Teknik Informatika", "angkatan": "2026", "status": "aktif" }, { "id": 4, "nim": "251004", "nama": "Nila Agustin Prima Aurelia",

			"email": "nilaaurelia97@gmail.com", "prodi_id": 2, "prodi": "Sistem Informasi", "angkatan": "2025", "status": "aktif" }, { "id": 3, "nim": "242003", "nama": "Citra Lestari", "email": "citra@email.com", "prodi_id": 2, "prodi": "Sistem Informasi", "angkatan": "2024", "status": "aktif" }, { "id": 2, "nim": "251002", "nama": "Ani Wijaya", "email": "ani@email.com", "prodi_id": 1, "prodi": "Teknik Informatika", "angkatan": "2025", "status": "aktif"

			}] }

GET	/mahasiswa?id=2		{
			"success": **true**,
			"data": {
			"id": 2,
			"nim": "251002",
			"nama": "Ani Wijaya",
			"email": "ani@email.com",
			"prodi_id": 1,
			"prodi": "Teknik Informatika",
			"angkatan": "2025",
			"status": "aktif"
			}
			}

POST	/mahasiswa	{
		"nim": "251009",
		"nama": "Ilma",
		"email": "ilma@email.com",
		"prodi_id": 1,
		"angkatan": 2025,
		"status": "aktif"
		}
			{
			"success": **true**,
			"message": "Data berhasil ditambahkan",
			"data": {
			"id": 8
			}
			}

PUT	mahasiswa?id=8	{ "nim": "251009", "nama": "Ilma Nur",	{ "success": **true**,

		"email": "ilma@email.com", "prodi_id": 1, "angkatan": 2025, "status": "aktif" }	"message": "Data berhasil diperbarui" }

DELETE	/mahasiswa?id=8		{
			"success": **true**,
			"message": "Data berhasil dihapus"
			}

GET	/buku		{ "success": **true**, "data": [{ "id": 2, "isbn": "9786020002", "judul": "Basis Data dengan MySQL", "penulis": "Rosa Ariani", "penerbit": "Andi", "tahun_terbit": "2021", "stok": 3 }, { "id": 1, "isbn": "9786020001",

			"judul": "Pemrograman PHP untuk Pemula", "penulis": "Budi Raharjo", "penerbit": "Informatika", "tahun_terbit": "2022", "stok": 5 }] }

GET	Buku?id=1		{
			"success": **true**,
			"data": {
			"id": 1,
			"isbn": "9786020001",
			"judul": "Pemrograman PHP untuk Pemula",
			"penulis": "Budi Raharjo",
			"penerbit": "Informatika",
			"tahun_terbit": "2022",
			"stok": 5
			}
			}

POST	/buku	{ "isbn": "8739020004",	{ "success": **true**, "message": "Data berhasil ditambahkan",

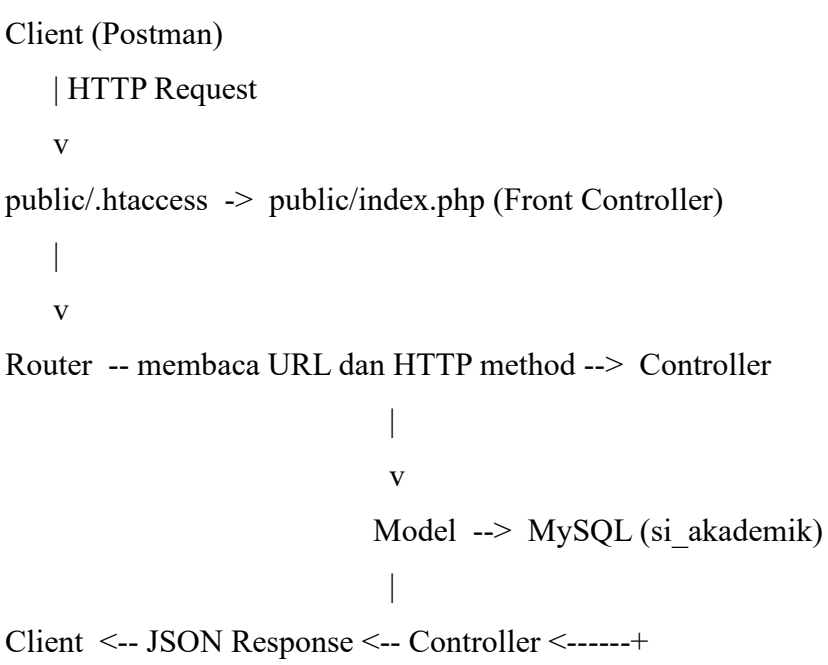
		"judul": "Dasar-dasar PHP",
		"penulis": "Nuril",
		"penerbit": "Gramedia",
		"tahun_terbit": "2016",
		"stok": 5
		}
			"data": {
			"id": 3
			}
			}

PUT	/buku?id=2	{
		"id": 2,
		"isbn": "9786020002",
		"judul": "Basis Data dengan MySQL untuk Pemula",
		"penulis": "Rosa Ariani",
		"penerbit": "Andi",
		"tahun_terbit": "2021",
		"stok": 3
		}
			{
			"success": **true**,
			"message": "Data berhasil diperbarui"
			}

DELETE	/buku?id=2		{
			"success": **true**,
			"message": "Data berhasil dihapus"
			}

Kode status error: 400 (data tidak valid), 404 (data atau endpoint tidak ditemukan), 405 (method tidak diizinkan), 409 (NIM atau ISBN duplikat), 500 (kesalahan koneksi database).

Diagram alur:



Bab 4. Kesimpulan

Praktikum Acara 17 berhasil membangun API CRUD data mahasiswa menggunakan PHP Native dengan arsitektur MVC. Berbeda dengan Acara 16 yang menempatkan seluruh proses dalam satu berkas, pada praktikum ini kode dipisahkan menurut tanggung jawabnya. Model (Mahasiswa dan Buku) menangani seluruh query ke database si_akademik dengan prepared statement PDO. Controller menerima request, memvalidasi data, memanggil Model, lalu mengembalikan response JSON dengan kode status HTTP yang sesuai. Router memetakan URL dan HTTP method ke Controller yang tepat, sedangkan autoloader spl_autoload_register() memuat class secara otomatis tanpa require manual. Seluruh request masuk melalui satu titik, yaitu Front Controller public/index.php yang dihubungkan oleh .htaccess, sehingga endpoint dapat diakses tanpa ekstensi .php.

Pengujian dengan Postman menunjukkan bahwa kelima operasi CRUD (POST, GET semua data, GET berdasarkan id, PUT, dan DELETE) berjalan sesuai alur Router, Controller, dan Model. Penggunaan kode status yang berbeda untuk tiap kondisi (200, 201, 400, 404, 405, dan 409) membuat hasil request mudah dipahami oleh client. Selain itu, validasi pada lapisan Controller dan Model terbukti mencegah data yang tidak layak masuk ke database, seperti field wajib yang kosong, format email yang salah, prodi yang tidak terdaftar, serta NIM atau ISBN yang sudah dipakai. Validasi NIM duplikat pada Tugas Mandiri 2 didukung pula oleh batasan UNIQUE KEY di tingkat database sebagai pengaman kedua.

Penambahan resource Buku pada Tugas Mandiri 1 menunjukkan bahwa struktur MVC ini mudah dikembangkan. Resource baru cukup dibuat dengan satu Model, satu Controller, dan

satu entri pada peta Router tanpa mengubah komponen lain. Dokumentasi endpoint dan diagram alur Router, Controller, Model yang disusun pada Tugas Mandiri 3 memperjelas hubungan antarlapisan tersebut